5th IEEE International Conference on AI in Cybersecurity (ICAIC), 18–20 February 2026, University of Houston, Houston, United States

Enterprise-Grade Security for the Model Context Protocol (MCP): Frameworks and Mitigation Strategies

Vineeth Sai Narajala
OWASP
New York, USA
vineeth.sai@owasp.com

Idan Habler
OWASP
Tel Aviv, Israel
edan.habler@gmail.com

2026 IEEE 5th International Conference on AI in Cybersecurity (ICAIC) | 978-1-6654-7761-1/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICAIC67076.2026.11395723

***Abstract*—The Model Context Protocol (MCP), introduced by Anthropic, provides a standardized framework for artificial intelligence (AI) systems to interact with external data sources and tools in real-time. While MCP offers significant advantages for AI integration and capability extension, it introduces novel security challenges that demand rigorous analysis and mitigation. This paper builds upon foundational research into MCP architecture and preliminary security assessments to deliver enterprise-grade mitigation frameworks and detailed technical implementation strategies. Through systematic threat modeling and analysis of MCP implementations and analysis of potential attack vectors, including sophisticated threats like tool poisoning, we present actionable security patterns tailored for MCP implementers and adopters. The primary contribution of this research lies in translating theoretical security concerns into a practical, implementable framework with actionable controls, thereby providing essential guidance for the secure enterprise adoption and governance of integrated AI systems.**

***Index Terms*—Model Context Protocol (MCP), AI Security, Zero Trust Architecture (ZTA), Tool Poisoning, Defense-in-Depth, Operational Security, Secure AI Integration, API Security, AI Governance**

I. Introduction

A. Background: The Model Context Protocol (MCP)

The Model Context Protocol (MCP) is a major step forward in standardizing how AI models interact with the world around them—giving them the ability to use tools and access real-time data on the fly [1]. Foundational research, like the work by Hou et al. [2], has laid out the basic components, how MCP works, and some early security concerns. Building on that, our focus shifts to what it takes to secure MCP in real-world, enterprise-level deployments.

To understand the security implications of MCP, it is essential to understand its fundamental architecture, which comprises three key components:

- **MCP Host:** Refers to the AI application or environment in which AI-driven tasks are performed and that operates the MCP client. Examples include applications such as Claude Desktop, and AI-driven integrated development tools like Cursor for software development. The host integrates tools and data, enabling interaction with external services through the MCP client and server.
- **MCP Client:** Serves as an intermediary in the host environment, facilitating communication between the MCP host and MCP servers. It sends requests and seeks information about the available services of servers. Data communication with servers is conducted securely and reliably via the transport layer.
- **MCP Server:** Serves as a gateway that allows the MCP client to interact with external services and execute tasks. The MCP Server offers three essential functionalities:
  - *Tools:* Allows the server to invoke external services and APIs to execute tasks on behalf of the AI model.
  - *Resources:* Exposes structured or unstructured datasets (e.g., local files, databases, and cloud platforms) to the model. When a model needs specific data (e.g., customer logs for a recommendation system), the server retrieves and processes it.
  - *Prompts:* Are reusable templates managed by the server to enhance model responses, maintain consistency, and simplify repeated actions.

As MCP moves from theory into production, strong and scalable security becomes critical. Standard API security practices remain important but are insufficient to address the unique risks associated with MCP's dynamic, tool-based model. One example is "tool poisoning"—a type of attack where maliciously crafted tool descriptions trick AI models into doing things they shouldn't [3]. Our goal in this paper is to go beyond identifying risks, offering actionable security strategies that enterprises can implement to safeguard their MCP systems.

B. The Broader Context: MCP Security and AI Governance

Securing protocols like MCP isn't just a technical issue—it's closely tied to the bigger picture of AI governance and responsible AI use. As AI systems become more autonomous and start interacting directly with external tools and real-time data through things like MCP, making sure those interactions are secure becomes absolutely essential. As a result, securing MCP is essential to building trustworthy AI systems, especially in complex enterprise environments. This paper contributes to that goal by providing practical strategies to

978-1-6654-7761-1/26/$31.00 ©2026 IEEE
Authorized licensed use limited to: BEN GURION UNIVERSITY. Downloaded on March 26,2026 at 06:12:43 UTC from IEEE Xplore. Restrictions apply.

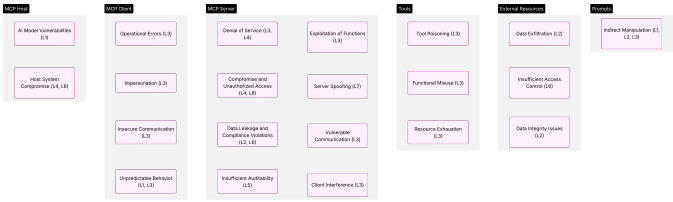

Fig. 1. This figure provides a comprehensive categorization of security threats across different components of the Model Context Protocol (MCP) architecture. The diagram organizes threats by component, with each component displayed as a column and specific threats listed within them.

mitigate the unique security risks introduced when AI systems interact with external tools.

C. Research Focus and Contributions

This paper moves beyond identifying potential MCP vulnerabilities to providing actionable security controls tailored for enterprise contexts. Our primary contributions are:

1) A Comprehensive, Multi-Layered Security Framework: Detailing a defense-in-depth strategy specifically adapted for the nuances of MCP deployments.
2) Actionable Zero Trust Implementation Patterns: Providing specific guidance on applying Zero Trust principles [4], [5] within MCP environments, focusing on identity, access, and continuous verification.
3) Advanced Threat Mitigation Techniques: Outlining sophisticated validation and detection methods to counter tool poisoning, data exfiltration, and manipulation attacks.
4) Operational Security Procedures: Defining best practices for ongoing monitoring, incident response, and threat intelligence integration specific to MCP.
5) Secure Reference Architectures: Illustrating practical deployment patterns for secure MCP integration within enterprise ecosystems.

While prior research [2] has mapped the MCP landscape and identified potential threat categories, our work distinguishes itself by delivering detailed, actionable security controls and configurations designed for immediate application by security practitioners and system architects responsible for hardening MCP implementations.

II. Security Threat Landscape and Methodology

A. Threat Modeling Methodology for MCP

Building upon our general risk identification approach [6], we employ the MAESTRO framework [7] for comprehensive threat modeling of AI systems as applied to MCP [8]. This framework provides a systematic methodology by examining potential vulnerabilities across seven specific layers of an AI system's architecture, allowing for a structured analysis of the MCP security landscape.

1) MAESTRO Framework for MCP: The MAESTRO framework examines AI system vulnerabilities across seven distinct layers, each relevant to different aspects of MCP security:

- L1 – Foundation Models: Concerns related to the underlying AI models, training data, and inherent vulnerabilities.
- L2 – Data Operations: Security of external data management and integration within MCP systems.
- L3 – Agent Frameworks: Vulnerabilities in agent logic, protocols, and tool utilization mechanisms.
- L4 – Deployment Infrastructure: Security of hardware and software environments hosting MCP components.
- L5 – Evaluation & Observability: Vulnerabilities in monitoring and diagnostic systems for MCP behavior.
- L6 – Security & Compliance: Issues related to access controls, policy enforcement, and regulatory requirements.
- L7 – Agent Ecosystem: Security challenges in interactions with humans, external tools, or other agents.

By applying this layered approach to MCP, we can systematically identify and categorize threats across the entire protocol stack, from underlying models to deployment infrastructure and ecosystem interactions. Fig. 1 illustrates the comprehensive threat categories.

2) Key Threat Categories by MCP Component: Using the MAESTRO framework, we identified the following key threats organized by MCP component:

a) MCP Server Threats:

- Compromise and Unauthorized Access: Vulnerabilities in deployment or security configurations that permit attacker access (L4, L6).
- Exploitation of Functions: Vulnerable tools potentially utilized to execute harmful operations (L3).
- Denial of Service: Server flooding through excessive requests or autonomous agent loops (L3, L4).
- Vulnerable Communication: Interception or modification of data between client and server (L3).
- Client Interference: Insufficient isolation allowing one client's behavior to affect others (L3).
- Data Leakage and Compliance Violations: Unauthorized data exfiltration or regulatory violations (L2, L6).
- Insufficient Auditability: Inadequate logging restricting detection and investigation of security events (L5).
- Server Spoofing: Malicious servers posing as legitimate ones within the ecosystem (L7).

b) MCP Client Threats:

- Impersonation: Attackers posing as legitimate clients to gain unauthorized access (L3).
- Insecure Communication: Vulnerabilities leading to interception or data alteration during transmission (L3).
- Operational Errors: Schema inconsistencies between client and server causing functional errors (L3).
- Unpredictable Behavior: Model instability prompting disruptive or unexpected requests (L1, L3).

c) MCP Host Environment Threats:

- AI Model Vulnerabilities: Hallucinations, instability, or specific exploits resulting in erroneous MCP utilization (L1).
- Host System Compromise: General compromise of the host system affecting MCP client security (L4, L6).

Authorized licensed use limited to: BEN GURION UNIVERSITY. Downloaded on March 26,2026 at 06:12:43 UTC from IEEE Xplore. Restrictions apply.

d) Data Sources and External Resources Threats:

- Insufficient Access Control: Inadequate security measures on underlying data sources enabling unauthorized access (L6).
- Data Integrity Issues: Data drift or intentional tampering leading to misleading results (L2).
- Data Exfiltration: Unauthorized extraction of confidential information through compromised connections (L2).

e) Tool-Related Threats:

- Functional Misuse: Tools utilized for improper or harmful purposes beyond intended functionality (L3).
- Resource Exhaustion: Excessive tool usage overwhelming system resources (L3).
- Tool Poisoning: Malicious manipulation of tool descriptions or parameters to induce unintended model behavior (L3).

f) Prompt-Related Threats:

- Indirect Manipulation: Attacks targeting the foundation model (e.g., prompt injection), influencing MCP utilization (L1, L2, L3).

This threat analysis informs our comprehensive security framework and mitigation strategies presented in Section III, enabling a defense-in-depth approach tailored to the specific security challenges of MCP implementations in enterprise environments.

B. Key Security Challenges

MCP introduces a distinct set of security challenges because it acts as a bridge between powerful AI models and often untrusted external tools and data sources. Its dynamic nature creates a complicated trust landscape that includes the MCP server, the AI model itself, client applications, and the various tools plugged into the system.

Building upon the threat categorization identified by Hou et al. [2]—covering installation, operational, and update-related risks—our focus is on mitigating enterprise-critical threats, including:

- Tool Poisoning: Malicious manipulation of tool descriptions or parameters to induce unintended or harmful actions by the AI model.
- Data Exfiltration: Unauthorized extraction of sensitive data through compromised tools or manipulated MCP responses.
- Command and Control (C2): Establishment of covert C2 channels via compromised MCP servers or tools.
- Update Mechanism Compromise: Insertion of persistent backdoors or malware through compromised MCP server or tool update channels.
- Identity and Access Control Subversion: Exploitation of authentication or authorization flaws to gain unauthorized access or escalate privileges.
- Denial of Service (DoS): Overloading MCP servers or dependent resources through excessive requests or resource exhaustion attacks.
- Insecure Configuration: Exploitation of misconfigurations in MCP servers, network settings, or access controls.

C. Methodology

The security framework and mitigation strategies presented in this paper were developed using the following methodology:

1) Risk Identification and Analysis: We identified potential security risks through a combination of:
   - Reviewing existing literature on MCP and related AI integration security challenges [3].
   - Analyzing the MCP protocol specification and its interaction model to identify potential weak points.
   - Applying established threat modeling concepts (drawing inspiration from frameworks like STRIDE - Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) to the MCP ecosystem.
   - Extrapolating threats from known vulnerabilities in analogous technologies like APIs, web services, and agent-based systems [9].
   - Conducting structured threat modeling of MCP architecture.

   Prioritization of risks was based on qualitative assessment of potential impact and likelihood, informed by the attack scenarios and known vulnerability patterns in similar systems.
2) Mitigation Strategy Development: We designed mitigation strategies by:
   - Synthesizing established security best practices from recognized sources, including National Institute of Standards and Technology (NIST) Special Publications (SP) 800-53 Rev 5 [10] (Security Controls) and SP 800-207 [4] (Zero Trust Architecture), Open Web Application Security Project (OWASP) guidelines [9], and general secure coding and threat modeling principles.
   - Adapting these general principles specifically to the unique characteristics of MCP, such as the dynamic nature of tool interaction and the potential for semantic manipulation (tool poisoning).
   - Structuring the mitigations around core security principles: defense-in-depth and Zero Trust.
   - Organizing controls across multiple layers: network, application gateway, container/host, identity/access, tool/prompt management, input/output validation, and operations.

III. Comprehensive MCP Security Framework

We propose a multi-layered security framework based on defense-in-depth and Zero Trust principles, tailored to the specific risks of MCP. The overall security framework (see Fig. 2) provides a high-level overview of the security framework.

A. MCP Server-Side Mitigations

1) Network Segmentation and Microsegmentation: Network segmentation is a fundamental security strategy that goes beyond traditional perimeter-based defenses. In MCP

Authorized licensed use limited to: BEN GURION UNIVERSITY. Downloaded on March 26,2026 at 06:12:43 UTC from IEEE Xplore. Restrictions apply.

environments, this approach is exponentially more critical due to the protocol's dynamic nature of tool interactions.

- **Dedicated MCP Security Zones:** Isolate MCP servers and critical components within dedicated network segments (e.g., Virtual Local Area Networks (VLANs), Virtual Private Cloud (VPC) subnets) with strict ingress/egress filtering rules based on the principle of least privilege. Use Security groups as well in Cloud Environments like AWS.
- **Service Mesh Implementation:** Employ a service mesh (e.g., Istio) to enforce fine-grained, identity-based traffic control (mutual Transport Layer Security - mTLS) between MCP microservices and connected tools, independent of network topology when using Kubernetes architecture.
- **Application-Layer Filtering Gateways:** Deploy gateways (e.g., Web Application Firewalls (WAFs), API Gateways) capable of deep packet inspection (DPI) for MCP traffic, configured with rules to detect protocol anomalies, malicious payloads in tool descriptions/parameters, and known attack signatures.
- **End-to-End Encryption:** Mandate TLS 1.2+ with strong cipher suites and certificate pinning for all MCP-related communication paths.

Rationale: Contains breaches, prevents lateral movement, and protects data in transit.

2) Application Gateway Security Controls: Application-level gateways inspecting MCP traffic should enforce:

- **Strict Protocol Validation:** Rigorously validate all MCP messages against the official protocol specification, rejecting any malformed or non-compliant requests.
- **Threat Detection Patterns:** Implement rules and heuristics to detect suspicious patterns indicative of tool poisoning, command injection, or data exfiltration attempts within tool descriptions, parameters, and responses. Create these rules based on real-world scenarios (example db: https://www.promptfoo.dev/lm-security-db).
- **Rate-Limiting and Anti-Automation:** Apply granular rate limiting based on source IP, authenticated user, and specific MCP endpoints to mitigate DoS and brute-force attacks. Employ bot detection mechanisms.
- **Comprehensive Request Tracing:** Implement distributed tracing (e.g., using OpenTelemetry) to maintain context across authentication, MCP interactions, and tool invocations for effective auditing and incident analysis.

Rationale: Provides protocol-specific defense against application-layer attacks.

3) Secure Containerization and Orchestration: Deploy MCP servers in hardened containerized environments only when a code execution environment is not present:

- **Immutable Infrastructure:** Utilize read-only container file systems, mounting writable volumes only where strictly necessary and monitored.
- **Restricted Capabilities:** Drop unnecessary Linux capabilities within containers (e.g., CAP_NET_ADMIN, CAP_SYS_ADMIN).
- **Resource Quotas:** Enforce strict CPU, memory, network I/O, and storage quotas to prevent resource exhaustion and contain misbehaving components.
- **Seccomp and AppArmor/SELinux:** Apply fine-grained seccomp (secure computing mode) profiles to restrict allowed system calls and use AppArmor or SELinux for mandatory access control (MAC) to further limit potential exploit impact.
- **Regular Scanning:** Integrate vulnerability scanning into the Continuous Integration/Continuous Deployment (CI/CD) pipeline for container images.

Rationale: Minimizes the attack surface and limits the blast radius of a compromised container.

4) Host-Based Security Monitoring: The goal is to create a comprehensive observability framework capable of detecting potential compromises that may evade higher-level security controls, providing a critical safety net in the complex landscape of AI tool integrations. Deploy Endpoint Detection and Response (EDR) or specialized Host-Based Intrusion Detection Systems (HIDS) on underlying hosts/VMs:

- **MCP-Specific Behavioral Rules:** Develop detection rules tailored to anomalous MCP server process behavior, file access patterns, network connections, or tool invocation sequences.
- **File Integrity Monitoring (FIM):** Continuously monitor critical MCP server binaries, configuration files, and tool definitions for unauthorized modifications.
- **Memory Analysis:** Employ techniques to detect in-memory attacks (e.g., code injection, hooking) that may not leave persistent file system traces.

Rationale: Provides visibility into potential compromises at the host level, detecting threats that bypass network or application controls.

5) Enhanced OAuth 2.0+ Implementation: Secure MCP server authorization using OAuth 2.0+ principles with enhancements:

- **Strong Client and User Authentication:** Mandate robust client authentication methods (e.g., mTLS, JSON Web Token (JWT) assertion) and require Multi-Factor Authentication (MFA) for user authentication, potentially using adaptive/risk-based authentication.
- **Fine-Grained, Scoped Access Tokens:** Issue short-lived access tokens with the narrowest possible scopes (permissions) required for the specific tool invocation, adhering strictly to the principle of least privilege. Consider Rich Authorization Requests (RAR) for more granular authorization data.
- **Audience Restriction:** Ensure tokens are audience-restricted (aud) to the specific MCP resource server or tool being accessed.
- **Sender-Constrained Tokens:** Implement mechanisms like Demonstration of Proof-of-Possession (DPoP) or

Authorized licensed use limited to: BEN GURION UNIVERSITY. Downloaded on March 26,2026 at 06:12:43 UTC from IEEE Xplore. Restrictions apply.

mTLS token binding to prevent token theft and replay attacks.
- **Regular Key Rotation:** Automate frequent rotation of keys used for token signing and encryption.

Rationale: Establishes strong, verifiable identity and authorization as the foundation for access decisions.

6) Tool and Prompt Security Management: Tools in the MCP ecosystem are dynamic, potentially executable entities with complex interaction patterns, rather than static code repositories. This section explores comprehensive strategies for vetting, validating, and monitoring tools throughout their lifecycle, treating each tool as a potential vector for sophisticated attacks such as tool poisoning, data exfiltration, and unauthorized system access.

a) Robust Tool Vetting and Onboarding: Implement a stringent process for approving and onboarding new tools:

- *Security Review Checkpoints:* Mandate security reviews, including static analysis (SAST), dynamic analysis (DAST), dependency scanning (Software Composition Analysis - SCA), and potentially manual code review/pentesting for complex or high-risk tools.
- *Clear Documentation:* Require comprehensive documentation detailing the tool's purpose, required permissions, data handling practices, interaction patterns, and security considerations.
- *Structured Approval Workflow:* Establish a formal approval process involving security, data governance, privacy, legal, and business stakeholders.
- *Periodic Recertification:* Require regular review and recertification of deployed tools to ensure ongoing compliance and security.

Rationale: Prevents the integration of inherently insecure or malicious tools.

b) Content Security Policy for Tool Descriptions: Treat tool descriptions as potentially executable content and apply controls:

- *Structured Validation and Sanitization:* Enforce a strict schema for tool descriptions. Sanitize descriptions rigorously, removing or encoding potentially active content (e.g., script tags, unexpected directives) and validating parameter constraints.
- *Malicious Pattern Detection:* Use pattern matching (Regular Expressions - RegEx, YARA rules) and semantic analysis to detect known malicious instruction patterns, command injection attempts, or data exfiltration syntax within descriptions.
- *Permission Validation:* Cross-reference requested permissions/scopes against the tool's described functionality, flagging excessive or suspicious requests.
- *Integrity Verification:* Implement cryptographic signing and verification for tool descriptions stored in registries to prevent tampering after approval.

Rationale: Directly mitigates the risk of tool poisoning attacks via manipulated descriptions.

c) Advanced Tool Behavior Monitoring and Poisoning Detection: Go beyond static analysis to detect malicious behavior at runtime:

- *Behavioral Baselining:* Establish normal operational baselines for each tool (e.g., typical resource usage, network connections, API calls, data access patterns) and alert on significant deviations.
- *Dynamic Analysis/Sandboxing:* Execute tools in isolated sandbox environments during testing or even for initial production requests to observe behavior and detect unexpected actions (e.g., unauthorized network calls, file system access).
- *AI/ML-Powered Detection:* Employ machine learning models trained on legitimate and malicious tool behaviors/descriptions to identify novel or obfuscated poisoning techniques.
- *Security Telemetry Correlation:* Correlate tool activity logs with broader security telemetry (network logs, endpoint logs, authentication logs) to identify components of sophisticated, multi-stage attacks.

Rationale: Detects sophisticated poisoning attacks that manifest during execution rather than in static descriptions.

B. MCP Client-Side Mitigations

1) Zero-Trust Security Model Implementation: The Zero Trust security model represents a paradigm shift from traditional perimeter-based security architectures. Rather than assuming trust based on network location, Zero Trust operates on the fundamental principle of "never trust, always verify." In the context of the Model Context Protocol, this approach becomes particularly critical due to the protocol's inherently dynamic and potentially unpredictable interaction model. The traditional notion of a trusted internal network becomes obsolete when dealing with AI systems that can interact with a vast, constantly changing ecosystem of tools and data sources. Zero Trust provides a comprehensive framework for continuously validating every access attempt, ensuring that each interaction is scrutinized regardless of its origin or apparent legitimacy. Assume no implicit trust; continuously verify every access attempt.

2) Just-in-Time (JIT) Access Provisioning: Just-in-Time (JIT) access provisioning challenges traditional resource access management by eliminating standing privileges. In the MCP ecosystem, where tools and resources can be dynamically invoked and may require temporary, highly specific access, JIT becomes a critical security mechanism. This approach minimizes the attack surface by dramatically reducing the window of potential unauthorized access.

- **Dynamic, Time-Limited Access:** Issue temporary access grants or credentials only for the duration required to complete a specific task or session.
- **Context-Aware Access Decisions:** Base access decisions not only on identity but also on context like device posture, location, time-of-day, and behavioral analytics.
- **Purpose-Driven Authorization:** Align requested permissions with the declared purpose of the tool invocation.

Authorized licensed use limited to: BEN GURION UNIVERSITY. Downloaded on March 26,2026 at 06:12:43 UTC from IEEE Xplore. Restrictions apply.

- **Real-Time Revocation:** Implement mechanisms for immediate revocation of access if risk factors change or suspicious activity is detected.
- For multi-step or multi-tool access, grant access to each tool only when necessary.

Rationale: Drastically reduces the window of opportunity for attackers by minimizing persistent access.

3) Continuous Validation and Monitoring: Continuous validation departs from traditional authentication models that rely on a single, static authentication event. In the complex and dynamic world of MCP interactions, security cannot be a one-time checkpoint but must be an ongoing, real-time process. Continuously reassess trust throughout interactions by:

- **Per-Request Authorization:** Re-validate authorization for every MCP request, not just at the beginning of a session.
- **Behavioral Anomaly Detection:** Apply User and Entity Behavior Analytics (UEBA) to MCP interactions, identifying deviations from established normal patterns for users and tools.
- **Risk-Based Authentication Step-Up:** Trigger requirements for additional authentication factors if the assessed risk level increases during a session (e.g., sensitive tool invocation, anomalous behavior).
- **Session Monitoring and Reassessment:** Continuously monitor session activity and periodically reassess trust based on changing context or risk signals.

Rationale: Ensures security posture adapts dynamically to changing conditions and potential threats emerging mid-session.

4) Cryptographic Verification of Tool Sources and Integrity: Ensure tools are authentic and unmodified:

- **Mandatory Code Signing:** Require all integrated tools (especially binaries or executables) to be cryptographically signed by trusted entities (internal Certificate Authority (CA) or reputable third parties). Verify signatures before execution.
- **Secure Tool Registry:** Maintain a secure, preferably immutable registry (e.g., using blockchain or signed Merkle trees), of approved tools, their versions, and corresponding cryptographic hashes or signatures.
- **Supply Chain Security:** Implement measures to secure the tool development and distribution pipeline (e.g., Supply chain levels for Software Artifacts (SLSA) framework) to prevent tampering before integration.

Rationale: Provides strong assurance of tool authenticity and integrity, preventing the execution of compromised or unauthorized code.

5) Input/Output Validation Framework: Rigorous validation of data flowing through MCP is critical.

a) Strict Schema Validation for MCP Messages: Enforce strict schema validation at MCP ingress points:

- *Data Type and Format Enforcement:* Validate data types, formats (e.g., regex for specific patterns), range constraints for numerical values, and enumerated values strictly using allowlists.
- *Length and Range Constraints:* Apply realistic length limits to strings and range constraints to numerical values to prevent buffer overflows and resource exhaustion.
- *Recursive Validation:* Ensure validation is applied recursively to nested data structures within MCP messages.
- *Reject Unknown Fields:* Configure validators to reject messages containing fields not defined in the schema to prevent parameter smuggling or pollution.

Rationale: Provides a fundamental defense against injection attacks and malformed requests.

b) Context-Aware Input Sanitization and Validation: Go beyond structural validation:

- *Contextual Escaping/Encoding:* Apply output encoding appropriate to the context where data will be used by the tool (e.g., HTML escaping, SQL parameterization). Note: This should primarily be handled at the tool level, but MCP can provide contextual guidance.
- *Input Normalization:* Normalize inputs (e.g., Unicode canonicalization, case normalization) before validation to prevent evasion techniques.
- *Semantic Validation:* Where possible, validate that input values are semantically meaningful within the context of the requested operation.
- *Cross-Field Consistency Checks:* Implement rules to check for logical consistency between different fields in a request.

Rationale: Defends against sophisticated injection and evasion techniques missed by basic schema validation.

6) Security Requirements for Hosting a Public Anthropic MCP Server: In order to host a public secure MCP Server, there's a need to establish a robust security framework that involves key security practices aimed at reducing the server's attack surface and restricting the capabilities of a malicious actor.

- **Agent Boundaries and State Isolation:** The sessions of individual agents must be isolated and kept separate from other sessions, preventing cross-session or cross-agent interference.
- **Input Validation and Sanitization:** Validate and sanitize inputs (e.g., user prompts, tool parameters) passed to the server in order to prevent prompt injection and parameter pollution attempts.
- **Operating System Hardening:** Deploy a minimal operating system comprising solely needed components and services. Strengthen the kernel by implementing appropriate security parameters, deactivating unnecessary kernel modules, and employing security-enhancing tools such as SELinux or AppArmor for required access control.
- **Service and Port Management:** Identify and deactivate all services and network ports that are not essential for the functioning of the MCP server. Conduct regular audits of active services to verify that only permitted services are operational, thus reducing vulnerability to threats.
- **Secure Remote Access:** Prohibit direct root login using SSH and require SSH key-based authentication. Alter

Authorized licensed use limited to: BEN GURION UNIVERSITY. Downloaded on March 26,2026 at 06:12:43 UTC from IEEE Xplore. Restrictions apply.

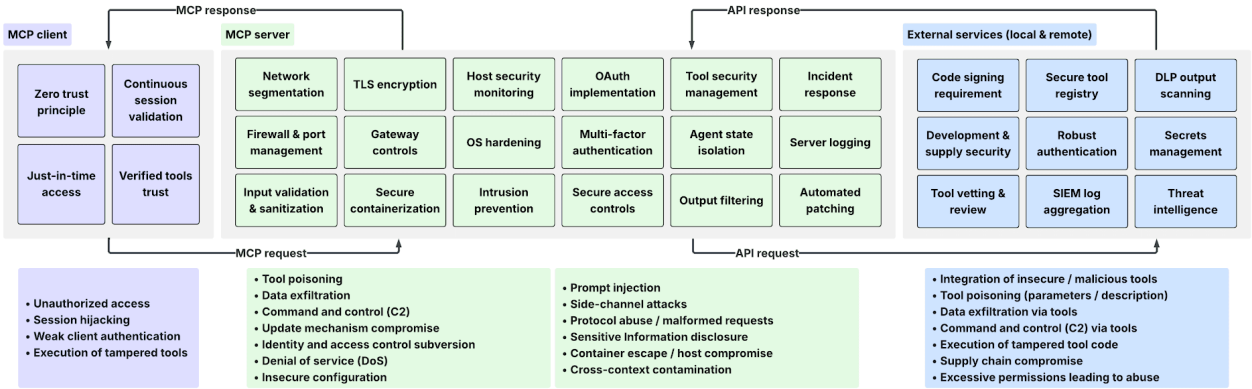

Fig. 2. This figure illustrates a multi-layered security framework for Model Context Protocol (MCP) based on defense-in-depth and Zero Trust principles.

the default SSH port to a non-standard designation and enforce IP allowlisting to limit SSH access to authorized networks. To enhance security, utilize a VPN for remote access from untrusted networks.

- **Strict Firewall Configurations:** Configure firewalls according to a "deny all by default" approach, periodically assess and refine firewall rules, and implement Web Application Firewalls (WAFs) for enhanced security.
- **Employ Intrusion Detection and Prevention Systems (IDPS):** Utilize IDPS to oversee network traffic and issue alerts for anomalous behavior.
- **Enforce MFA for High-Privilege Access:** Mandate MFA for administrator logins and access to sensitive systems, employing mechanisms such as authenticator applications or hardware tokens.

7) Security Requirements for Multi-MCP Server Deployments: As organizations deploy multiple MCP servers to enhance AI capabilities across their ecosystem, specific security considerations become essential. Multi-server deployments introduce unique challenges related to isolation, authorization, and trust boundaries that must be addressed through a comprehensive security framework.

a) Hosting Requirements:

- *Containerized Deployment:* Implement Docker-based containerization for MCP servers to ensure consistent security configurations across deployments and prevent environment conflicts as highlighted by Docker's collaboration with Anthropic, which addresses challenges like dependency conflicts, cross-platform consistency, and complex setup requirements.
- *Secure Remote Hosting:* When using cloud-based or remote MCP server hosting, implement dedicated security controls that isolate server instances and enforce strict access boundaries between environments similar to Cloudflare's approach for remote MCP server hosting with built-in OAuth support and enhanced accessibility [11].
- *Secrets Management:* Implement a centralized secrets management system for managing authentication credentials across multiple MCP servers using a solution that injects API keys or other secrets on-the-fly rather than storing them in files that could be leaked.
- *Resource Isolation:* Configure resource quotas and limitations for each MCP server to prevent resource exhaustion attacks from compromising the availability of other servers in the deployment.

b) MCP Server Isolation:

- *Network-Level Segmentation:* Implement strict network policies that limit communication between MCP servers and enforce explicit allow rules for necessary interactions to prevent lateral movement in the event one server is compromised, as MCP servers represent high-value targets storing authentication tokens for multiple services.
- *Session State Isolation:* Ensure complete isolation of agent sessions between different MCP servers to prevent cross-session interference or data leakage, preventing scenarios where one compromised server could potentially access data from another server's sessions.
- *Environmental Separation:* Deploy distinct MCP servers for different security contexts (e.g., development, testing, production) with no shared authentication or access mechanisms between environments.

c) Trust and Safety:

- *Capability Discovery Controls:* Implement controls around capability discovery to ensure MCP clients can only discover and access authorized tools from approved servers similar to Cloudflare's MCPClientManager implementation that handles capability discovery with proper namespacing when connecting to multiple MCP servers [11].
- *Server Authentication:* Implement cryptographic verification of MCP server identities to prevent server spoofing attacks where malicious servers could pose as legitimate ones, mitigating risks where attackers could create their own MCP server instances using stolen credentials.
- *Tool Validation Framework:* Establish a centralized validation framework for approving and auditing tools across

Authorized licensed use limited to: BEN GURION UNIVERSITY. Downloaded on March 26,2026 at 06:12:43 UTC from IEEE Xplore. Restrictions apply.

all MCP servers, with stricter validation requirements for tools with elevated privileges or access to sensitive data.
- *Cross-Server Monitoring:* Implement coordinated security monitoring across all MCP server instances to detect patterns of suspicious activity that may span multiple servers or clients.

IV. Limitations and Implementation Challenges

While the proposed framework provides a comprehensive approach, organizations should be aware of inherent limitations and potential implementation challenges:

- **Complexity:** Implementing the full suite of controls requires significant security expertise, potentially new tooling investments, and ongoing operational resources.
- **Performance Overhead:** Certain security measures, such as deep packet inspection, complex cryptographic operations (e.g., DPoP), and intensive real-time monitoring, can introduce latency or performance overhead that needs careful management, especially in low-latency applications.
- **Tool Ecosystem Maturity and Vetting:** The effectiveness of the framework relies partially on the security posture of third-party tools integrated via MCP. Vetting external tools thoroughly can be challenging and resource-intensive, and the security quality of available tools may vary significantly.
- **Dynamic Threat Landscape:** AI models, tools, and attack techniques evolve rapidly. Security controls must be continuously reviewed, updated, and adapted to remain effective against emerging threats.
- **Empirical Validation Gap:** As a relatively new protocol, there is limited large-scale, publicly available data on real-world MCP attacks and the measured effectiveness of specific countermeasures. The validation in this paper is primarily conceptual and scenario-based.
- **Usability and Developer Experience:** Overly stringent security controls, if not implemented carefully, could potentially hinder developer productivity or negatively impact the usability of AI applications relying on MCP. Finding the right balance is crucial.
- **Policy Enforcement Consistency:** Ensuring consistent application of fine-grained security policies across a potentially large and diverse set of tools and MCP instances can be operationally complex.

Acknowledging these challenges is crucial for realistic planning and successful implementation of MCP security.

V. Conclusion

The Model Context Protocol offers powerful capabilities for extending AI systems but introduces significant security challenges that require proactive and sophisticated mitigation. Simply adopting standard API security practices is insufficient. This paper has presented a comprehensive, multi-layered security framework specifically tailored for MCP, emphasizing defense-in-depth, Zero Trust principles, rigorous tool vetting, continuous monitoring, and robust input/output validation.

We provided detailed implementation strategies, operational guidelines, and reference patterns designed to be actionable for security practitioners building or managing MCP deployments in enterprise environments. The framework aids prioritization through its structure and the inclusion of qualitative risk assessments. While the threat landscape will continue to evolve, and implementation presents challenges (Section V), implementing the framework described herein—integrating network, application, host, data, and identity controls—provides a strong foundation for securely leveraging MCP.

Organizations must treat MCP security as a critical priority from the outset of any implementation, integrating it into their overall AI governance strategy. By adopting a security-first mindset, implementing robust technical controls, establishing strong operational practices, acknowledging limitations, and staying abreast of emerging threats and research, enterprises can confidently harness the transformative potential of MCP while effectively managing the associated risks.

References

[1] Anthropic, "Introducing the model context protocol (mcp)," Anthropic Developer Documentation, 2024, accessed: 2025-04-10. [Online]. Available: https://docs.anthropic.com/claude/protocols/mcp/overview

[2] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model context protocol (mcp): Landscape, security threats, and future research directions," *arXiv preprint arXiv:2503.23278*, 2025. [Online]. Available: https://arxiv.org/abs/2503.23278

[3] D. Goodin, "Poisoned ai tools can leak secrets and execute malicious commands," Ars Technica, February 2024, accessed: 2025-04-10. [Online]. Available: https://arstechnica.com/security/2024/02/poisoned-ai-tools-can-leak-secrets-and-execute-malicious-commands/

[4] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," National Institute of Standards and Technology, Tech. Rep. Special Publication 800-207, 2020. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-207

[5] J. Kindervag, "Build security into your network's dna: The zero trust network architecture," Forrester Research, 2010.

[6] N. V. Sai and O. Narayan, "Securing agentic AI: A comprehensive threat model and mitigation framework for generative AI agents," 2025. [Online]. Available: https://arxiv.org/abs/2504.19956

[7] Cloud Security Alliance, "Maestro (multi-agent environment, security, threat, risk, & outcome): A novel threat modeling framework for agentic ai," Cloud Security Alliance Blog, February 2025, accessed: 2025-04-10. [Online]. Available: https://cloudsecurityalliance.org/blog/2025/02/06/agentic-ai-threat-modeling-framework-maestro

[8] K. Huang, A. Sheriff, J. Sotiropoulos, R. F. Del, and V. Lu, "Multi-agentic system threat modelling guide OWASP GenAI security project," Apr. 2025. [Online]. Available: https://www.researchgate.net/publication/391204915_Multi-Agentic_system_Threat_Modelling_Guide_OWASP_GenAI_Security_Project

[9] OWASP Foundation, "Announcing the owasp llm and gen ai security project initiative for securing agentic applications," OWASP Foundation, 2024, accessed: 2025-04-10.

[10] National Institute of Standards and Technology, "Ai 100-2 e2023 adversarial ml - a taxonomy of threats and mitigations," National Institute of Standards and Technology, Technical Report AI 100-2 e2023, 2023.

[11] SSOJet, "Cloudflare supports remote hosting for anthropic's mcp servers - ssojet," SSOJet Blog, April 2025, accessed: 2025-04-10. [Online]. Available: https://ssojet.com/blog/cloudflare-supports-remote-hosting-for-anthropics-mcp-servers/

Authorized licensed use limited to: BEN GURION UNIVERSITY. Downloaded on March 26,2026 at 06:12:43 UTC from IEEE Xplore. Restrictions apply.